

Sistemas Embebidos

Laboratorio 2: Desarrollo de AI y Control Web

Ejercicio 2.4: Análisis

5 de Mayo, 2026

1. Ejercicio 2.4: Análisis

En este ejercicio se analizaron las cinco topologías de redes neuronales entrenadas previamente, comparando su desempeño en términos de accuracy, tamaño, FPS estimado, potencia y energía consumida. Además, se construyó un *Roofline plot* para analizar el comportamiento estimado de las capas de las redes en función de su intensidad aritmética y el rendimiento máximo esperado en la ESP32.

Las redes comparadas fueron:

- `red_1_mlp_pequena`
- `red_2_mlp_profunda`
- `red_3_cnn_tiny`
- `red_4_cnn_small`
- `red_5_cnn_depthwise`

El objetivo principal de este análisis es decidir qué red resulta más conveniente para una posible implementación en ESP32, considerando no solo el accuracy, sino también las restricciones propias de un sistema embebido, como memoria, velocidad de inferencia y consumo energético.

1.1. Datos utilizados para el análisis

Para este análisis se utilizaron los resultados obtenidos en los ejercicios anteriores. Desde el entrenamiento de los modelos se obtuvo el accuracy, el tamaño del modelo cuantizado en formato `TFLite int8` y el número estimado de operaciones MAC. Desde la estimación en ESP32 se obtuvieron los FPS estimados, la latencia y la energía por inferencia.

En la ESP32 se midieron las siguientes operaciones base:

Cuadro 1: Mediciones base obtenidas en ESP32.

Medición	Valor
MAC float32	0.137650000 μ s/MAC
MAC int8	0.100097500 μ s/MAC
memcpy	0.003914642 μ s/byte
Memoria lectura + escritura	0.056621546 μ s/byte movido

A partir de estas mediciones se obtuvo el siguiente rendimiento aproximado:

Cuadro 2: Rendimiento aproximado medido en ESP32.

Métrica	Valor
MAC float32 por segundo	7,264,802.03 MAC/s
MAC int8 por segundo	9,990,259.50 MAC/s
Ancho de banda memcpy	243.62 MB/s
Ancho de banda memoria lectura + escritura	16.84 MB/s

1.2. Resumen de resultados por red

La Tabla 3 resume los resultados principales utilizados para comparar las redes. Se reporta el accuracy de presencia, el accuracy de cuadrante, el tamaño del modelo cuantizado en `int8`, el número de operaciones MAC, los FPS estimados, la energía por inferencia y la potencia activa considerada.

Cuadro 3: Resumen de resultados de las cinco redes.

Modelo	Acc. presencia	Acc. cuadrante	Tamaño int8 [bytes]	MACs	FPS	Energía [mJ]	Potencia [W]
red_1_mlp_pequena	89.62 %	53.77 %	1,577,352	1,573,024	4.01	98.822	0.396
red_2_mlp_profunda	89.62 %	53.77 %	6,312,552	6,301,856	1.01	392.440	0.396
red_3_cnn_tiny	89.62 %	53.77 %	7,200	1,179,984	8.24	48.037	0.396
red_4_cnn_small	90.57 %	59.43 %	26,776	14,746,784	0.68	586.245	0.396
red_5_cnn_depthwise	89.62 %	53.77 %	16,408	1,375,480	7.07	55.992	0.396

Para la estimación de potencia se utilizó:

$$P = V \cdot I$$

donde se consideró un voltaje de 3.3 V y una corriente activa estimada de 0.120 A. Por lo tanto:

$$P = 3,3 \cdot 0,120 = 0,396 \text{ W}$$

La energía por inferencia fue calculada como:

$$E = P \cdot t$$

donde t corresponde a la latencia total estimada de cada red. Por esto, aunque la potencia activa considerada es la misma para todas las redes, la energía consumida por inferencia cambia según la duración de la inferencia.

1.3. Accuracy vs tamaño

El primer gráfico solicitado compara el accuracy de las redes con el tamaño del modelo cuantizado en `int8`. Para este análisis se utilizó el accuracy de cuadrante, ya que esta salida representa el problema más difícil: no solo detectar si el identificador está presente, sino también localizarlo dentro de la imagen.

Además, se marcó una línea vertical correspondiente al tamaño máximo considerado para la ESP32. En este caso se usó como referencia una partición de aplicación de aproximadamente 1 MB. Este límite permite visualizar qué modelos podrían ser más problemáticos para una implementación embebida directa.

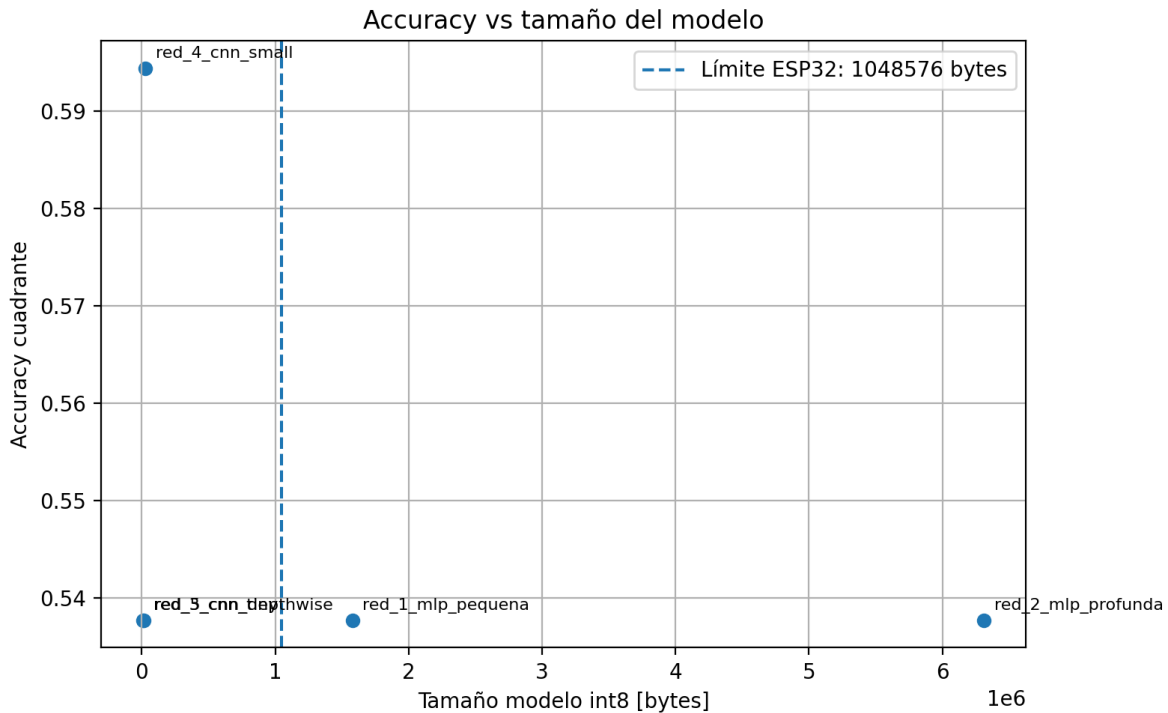


Figura 1: Accuracy de cuadrante versus tamaño del modelo cuantizado en formato `TFLite int8`.

En la Figura 1 se observa que las redes MLP tienen un tamaño considerablemente mayor que las redes convolucionales. La red `red_1_mlp_pequena` alcanza aproximadamente 1.58 MB, mientras que `red_2_mlp_profunda` supera los 6.31 MB. Ambas redes quedan por encima del límite conservador de 1 MB considerado para la ESP32.

En cambio, las redes convolucionales presentan tamaños mucho menores. `red_3_cnn_tiny` ocupa solo 7.2 KB, `red_5_cnn_depthwise` ocupa 16.4 KB y `red_4_cnn_small` ocupa 26.8 KB. Esto muestra que las redes convolucionales son mucho más adecuadas para el contexto embebido desde el punto de vista de almacenamiento.

La red `red_4_cnn_small` obtiene el mayor accuracy de cuadrante, con 59.43%, pero sigue siendo pequeña en términos de tamaño del modelo. Por lo tanto, desde la perspectiva exclusiva de accuracy versus tamaño, esta red parece una alternativa atractiva.

1.4. Accuracy vs FPS estimado

El segundo gráfico compara el accuracy de cuadrante con los FPS estimados para cada red. Los FPS fueron calculados a partir de la latencia total estimada en la ESP32, usando:

$$FPS = \frac{1}{t}$$

donde t corresponde al tiempo total estimado por inferencia en segundos.

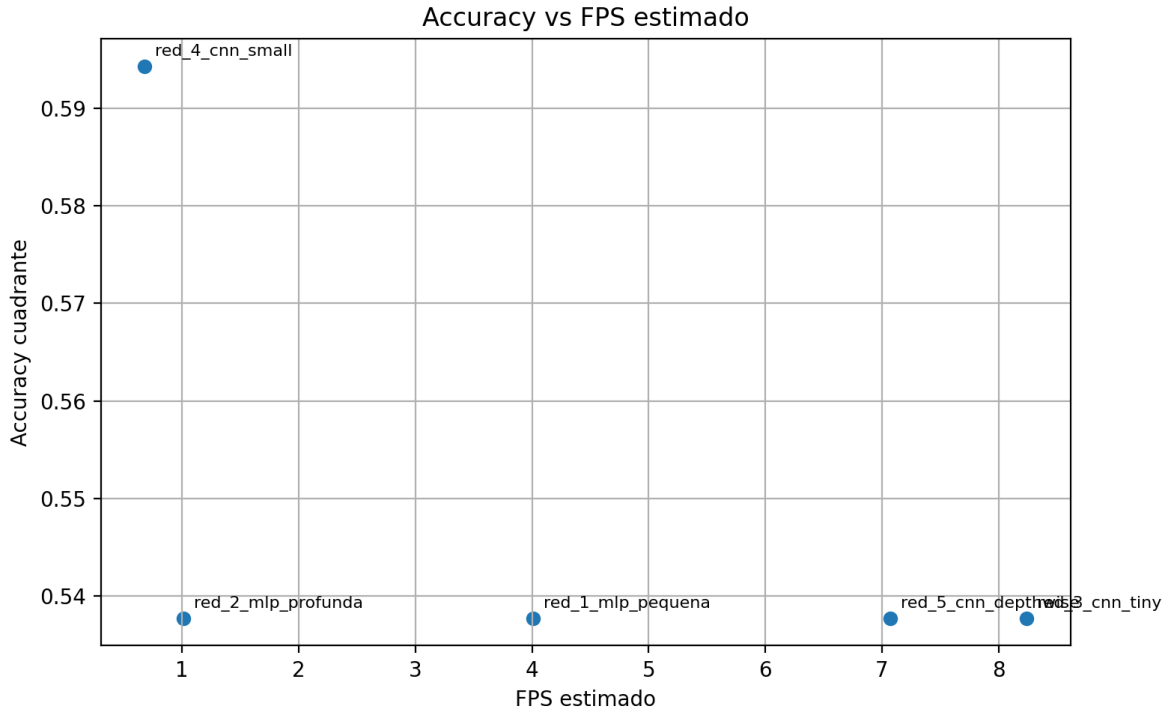


Figura 2: Accuracy de cuadrante versus FPS estimado.

En la Figura 2 se observa una diferencia importante entre las redes. La red **red_3_cnn_tiny** alcanza el mayor rendimiento estimado, con 8.24 FPS. Luego aparece **red_5_cnn_depthwise**, con 7.07 FPS. Ambas redes tienen un tamaño pequeño y un número relativamente bajo de operaciones.

La red **red_4_cnn_small**, aunque obtuvo el mejor accuracy de cuadrante, alcanza solo 0.68 FPS. Esto se debe a que requiere aproximadamente 14.75 millones de MACs, muy por encima de las otras redes convolucionales. Por lo tanto, aunque es la red más precisa, su velocidad estimada no es conveniente para una aplicación que requiera reacción rápida, como un robot sumo.

Las redes MLP tampoco resultan ideales. **red_1_mlp_pequena** alcanza 4.01 FPS y **red_2_mlp_profunda** solo 1.01 FPS. Además, ambas tienen un tamaño mucho mayor que las redes convolucionales.

1.5. Accuracy vs potencia consumida

El tercer gráfico compara el accuracy con la potencia consumida. En este caso, la potencia activa estimada fue la misma para todas las redes:

$$P = 0,396 \text{ W}$$

Esto ocurre porque se usó el mismo voltaje y la misma corriente activa estimada para la ESP32. Por lo tanto, el gráfico de accuracy versus potencia no muestra diferencias entre redes en el eje de potencia. Sin embargo, esto no significa que todas consuman lo mismo en términos prácticos, ya que cada red tiene una latencia distinta.

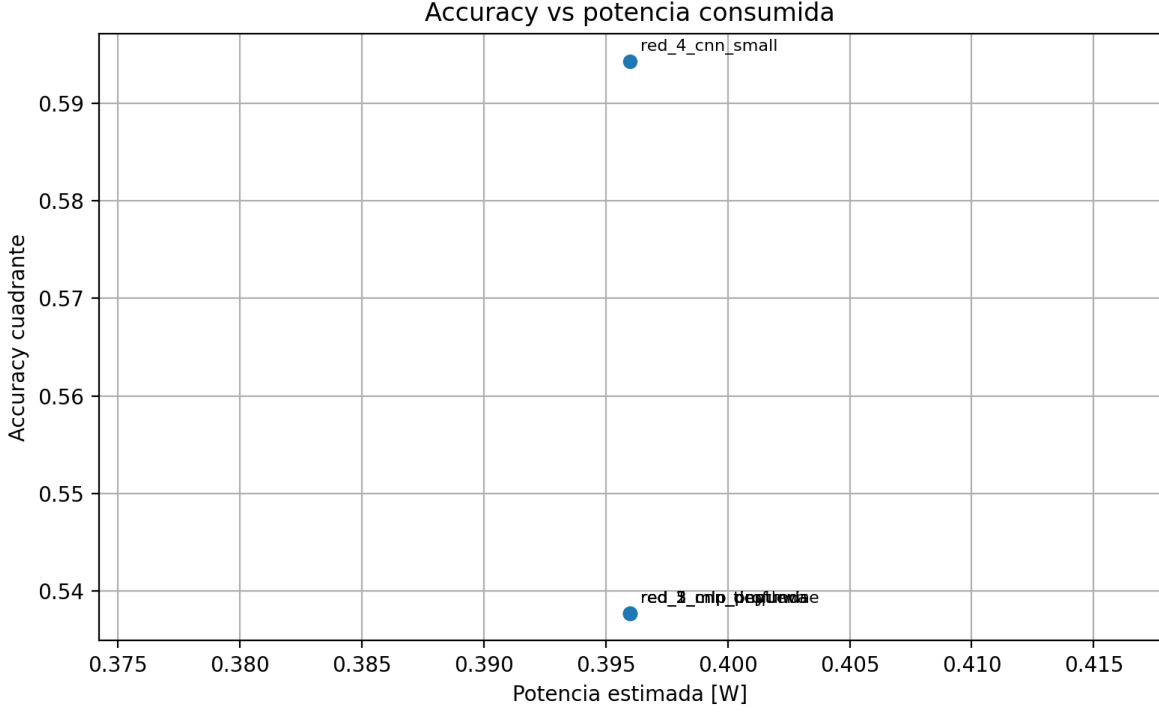


Figura 3: Accuracy de cuadrante versus potencia activa estimada.

En la Figura 3 se observa que todos los modelos se ubican sobre la misma potencia activa estimada. Esto se debe a que la estimación considera que la placa trabaja bajo el mismo estado de operación durante la inferencia. La diferencia real entre modelos aparece al considerar la energía consumida por inferencia, ya que una red que demora más tiempo mantiene la placa activa durante más tiempo.

1.6. Accuracy vs energía por inferencia

Aunque el enunciado solicita accuracy versus potencia consumida, también se generó el gráfico de accuracy versus energía por inferencia, ya que este permite comparar mejor el impacto real de la latencia de cada modelo. La energía por inferencia se calculó mediante:

$$E = P \cdot t$$

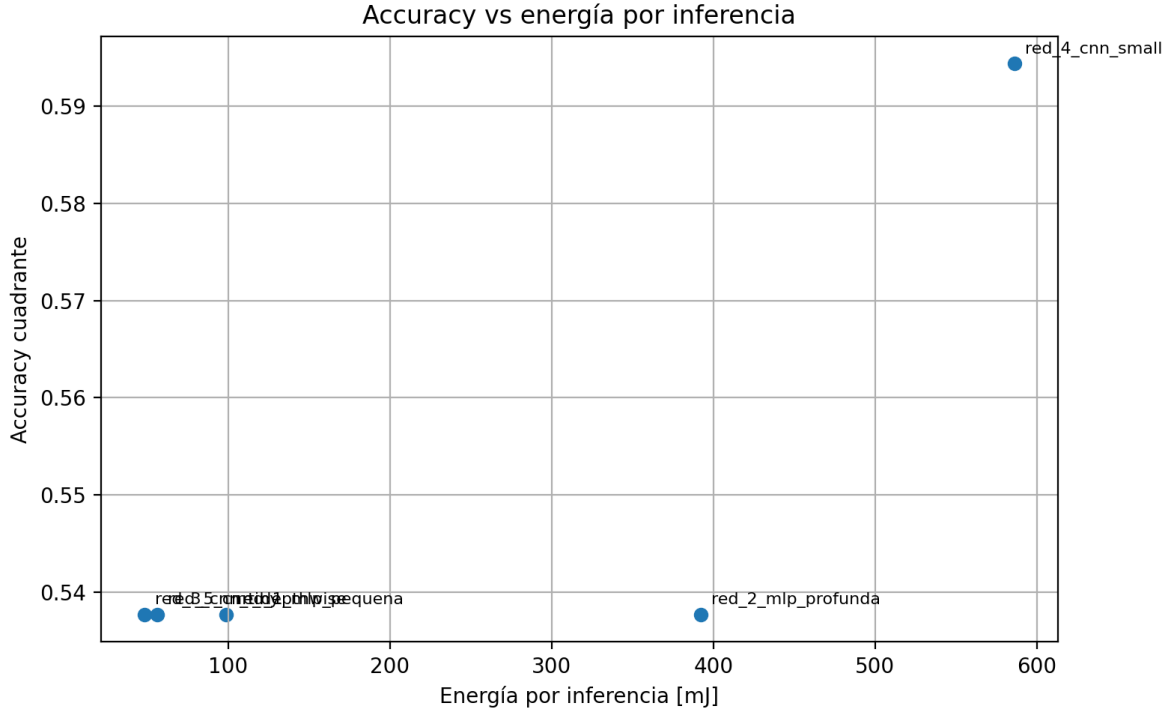


Figura 4: Accuracy de cuadrante versus energía estimada por inferencia.

En la Figura 4 se observa que **red_3_cnn_tiny** presenta el menor consumo energético por inferencia, con 48.037 mJ. Le sigue **red_5_cnn_depthwise**, con 55.992 mJ. Estas dos redes son las más eficientes desde el punto de vista energético.

La red **red_4_cnn_small**, aunque obtiene el mayor accuracy de cuadrante, consume aproximadamente 586.245 mJ por inferencia. Esto se debe a su alta latencia estimada, producto de su mayor cantidad de operaciones. Por lo tanto, su mejor accuracy viene acompañado de un costo energético considerablemente mayor.

La red **red_2_mlp_profunda** también presenta un consumo energético alto, de 392.440 mJ por inferencia, sin mejorar el accuracy respecto de modelos más pequeños. Esto refuerza la idea de que las MLP no son convenientes para esta aplicación embebida.

1.7. Roofline plot

El *Roofline plot* permite analizar el rendimiento estimado de las capas de las redes en función de su intensidad aritmética. La intensidad aritmética se define como:

$$I = \frac{\text{MACs}}{\text{bytes movidos}}$$

donde los bytes movidos consideran entradas, salidas y pesos de cada capa. El rendimiento estimado por capa se calculó usando el menor valor entre el límite computacional medido y el límite impuesto por el ancho de banda de memoria:

$$R = \min(R_{\text{peak}}, B \cdot I)$$

donde R_{peak} corresponde al rendimiento máximo medido en operaciones MAC int8 por segundo, y B corresponde al ancho de banda de memoria medido para lectura y escritura.

En este caso se utilizaron los siguientes valores medidos:

$$R_{\text{peak}} = 9,990,259,50 \text{ MAC/s}$$

$$B = 16,84 \text{ MB/s}$$

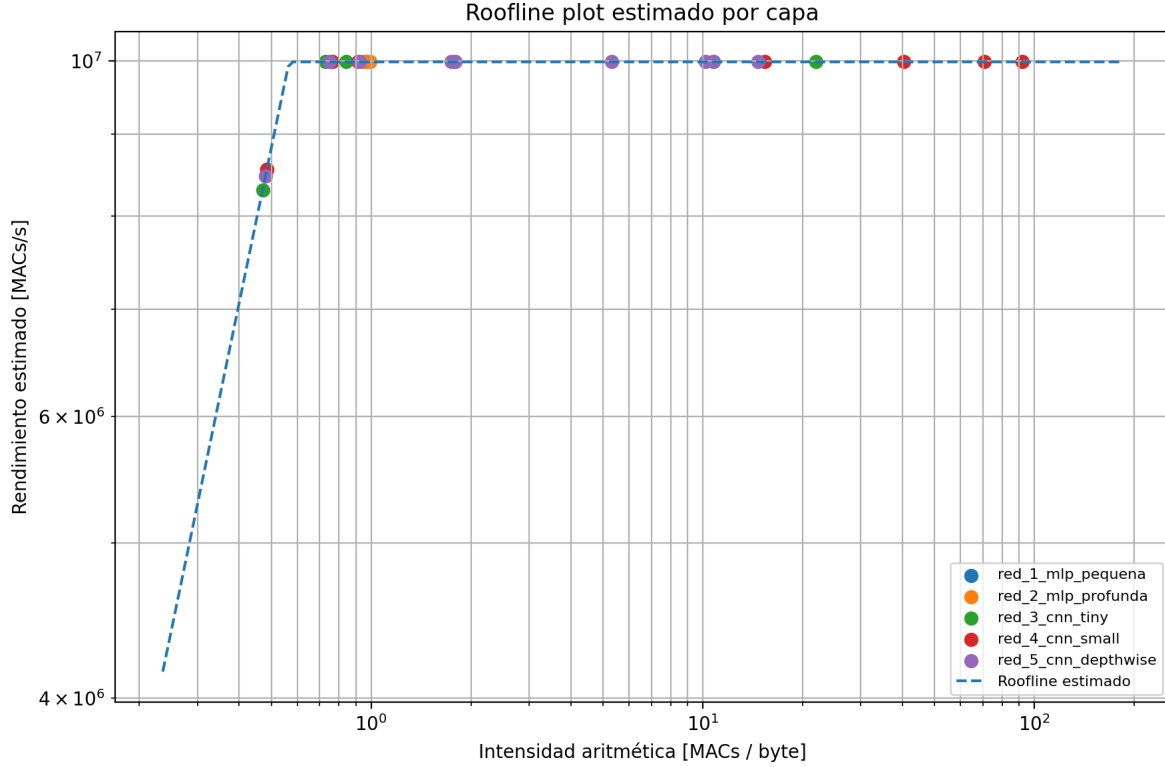


Figura 5: Roofline plot estimado para las capas de las cinco redes.

En la Figura 5 se muestra un único *Roofline plot* para todas las capas de las cinco redes. Cada punto representa una capa de una red. Las capas con baja intensidad aritmética tienden a estar más limitadas por memoria, mientras que las capas con mayor intensidad aritmética se acercan al límite computacional de la ESP32.

Las capas densas pequeñas de salida, como las capas **presencia** y **cuadrante**, tienen menor cantidad de operaciones y menor intensidad aritmética. Por otro lado, varias capas convolucionales presentan intensidades aritméticas mayores, ya que reutilizan pesos sobre múltiples posiciones espaciales de la imagen. Esto explica por qué las CNN suelen ser más adecuadas para procesamiento de imágenes en comparación con redes densas que simplemente aplanan toda la entrada.

En las redes MLP se observa que las capas densas principales poseen una gran cantidad de parámetros y operaciones, especialmente en **red_2_mlp_profunda**, donde la primera capa densa concentra

la mayoría del costo computacional. En contraste, `red_3_cnn_tiny` y `red_5_cnn_depthwise` distribuyen el costo en capas convolucionales más pequeñas, manteniendo un tamaño reducido y un buen rendimiento estimado.

1.8. Análisis comparativo

Al comparar los resultados, se observa que ninguna red domina completamente en todas las métricas. La elección depende del criterio que se priorice. Si se considera únicamente el accuracy, la mejor red es `red_4_cnn_small`, ya que alcanza 90.57 % de accuracy en presencia y 59.43 % en cuadrante. Sin embargo, esta red también tiene el mayor número de operaciones entre las CNN y el menor FPS estimado de todas las redes evaluadas, con solo 0.68 FPS.

Si se prioriza velocidad, la mejor red es `red_3_cnn_tiny`, que alcanza 8.24 FPS. Además, es la red más pequeña, con solo 7.2 KB en formato `TFLite int8`. Su energía por inferencia también es la menor, con 48.037 mJ. Aunque su accuracy de cuadrante es menor que el de `red_4_cnn_small`, su eficiencia la hace mucho más adecuada para una implementación en tiempo real.

La red `red_5_cnn_depthwise` también es una alternativa razonable. Tiene un tamaño pequeño de 16.4 KB, alcanza 7.07 FPS y consume 55.992 mJ por inferencia. Sin embargo, no logró superar a `red_3_cnn_tiny` en accuracy ni en velocidad. Por lo tanto, aunque su arquitectura *depthwise* es interesante para TinyML, en este caso no representa la mejor opción final.

Las redes MLP son las menos convenientes para la aplicación embebida. `red_1_mlp_pequena` tiene un tamaño de 1.58 MB y `red_2_mlp_profunda` alcanza 6.31 MB, sin mejorar el accuracy respecto de redes mucho más pequeñas. Además, `red_2_mlp_profunda` alcanza solo 1.01 FPS y consume 392.440 mJ por inferencia. Esto muestra que aplanar la imagen completa genera un costo alto y poco eficiente.

1.9. Decisión de red a utilizar

Considerando el objetivo de implementar una red neuronal en una ESP32 para detección del identificador, se decide utilizar la red:

`red_3_cnn_tiny`

La decisión se basa en que esta red ofrece el mejor equilibrio entre tamaño, velocidad y consumo energético. Aunque no alcanza el mejor accuracy de cuadrante, su desempeño es similar al de otras redes y su eficiencia es claramente superior.

Los principales argumentos para elegir `red_3_cnn_tiny` son:

- Es la red más pequeña, con solo 7,200 bytes en formato `TFLite int8`.
- Alcanza el mayor FPS estimado, con 8.24 FPS.
- Tiene el menor consumo energético por inferencia, con 48.037 mJ.
- Mantiene un accuracy de presencia de 89.62 %, similar al de las demás redes.
- Requiere aproximadamente 1.18 millones de MACs, menos que la mayoría de las alternativas.

- Su arquitectura convolucional es más apropiada para imágenes que una red MLP.

La red `red_4_cnn_small` podría ser elegida si el objetivo principal fuera maximizar el accuracy sin considerar la latencia. Sin embargo, en una aplicación embebida móvil, como un robot sumo, la velocidad de respuesta es muy importante. Una red con 0.68 FPS podría reaccionar demasiado lento frente a cambios rápidos del entorno.

Por esta razón, la red `red_3_cnn_tiny` es la opción más conveniente para continuar hacia una implementación en ESP32.

1.10. Conclusión

En este análisis se compararon cinco topologías de redes neuronales usando accuracy, tamaño del modelo, FPS estimado, potencia, energía e intensidad aritmética por capa. Los resultados muestran que las redes convolucionales son más adecuadas que las redes densas para este problema, principalmente porque logran tamaños mucho menores y mejor eficiencia computacional.

La red `red_4_cnn_small` obtuvo el mejor accuracy, especialmente en la clasificación de cuadrante. Sin embargo, su alto número de operaciones genera una latencia estimada elevada, reduciendo su rendimiento a 0.68 FPS y aumentando su energía por inferencia a 586.245 mJ.

La red `red_3_cnn_tiny` obtuvo el mejor balance general. Aunque su accuracy de cuadrante fue menor que el de `red_4_cnn_small`, alcanzó el mayor FPS, el menor tamaño y el menor consumo energético estimado. Por lo tanto, se selecciona `red_3_cnn_tiny` como la red más adecuada para una futura implementación en ESP32.